



Java Database Connectivity

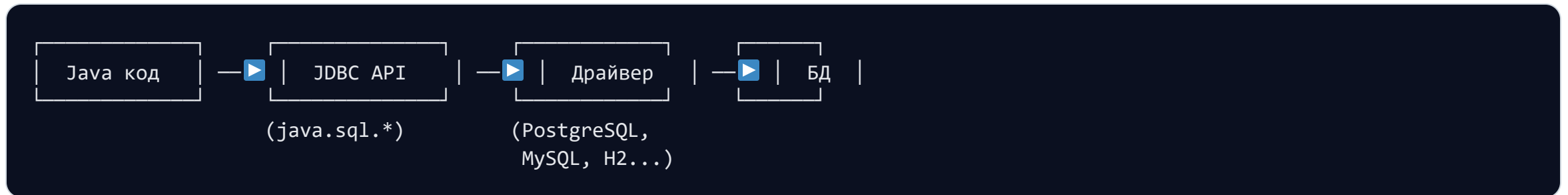
Блок 2, Часть 3

Цели лекции

После этой лекции ты будешь уметь:

-  Подключаться к БД через **JDBC**
-  Выполнять **CRUD** операции
-  Использовать **PreparedStatement** (защита от SQL injection)
-  Работать с **транзакциями**
-  Выполнять **batch** операции
-  Понимать типичные ошибки и **anti-patterns**

Архитектура JDBC



Ключевая идея:

Один API — разные базы данных. Меняешь драйвер — код остаётся тем же!

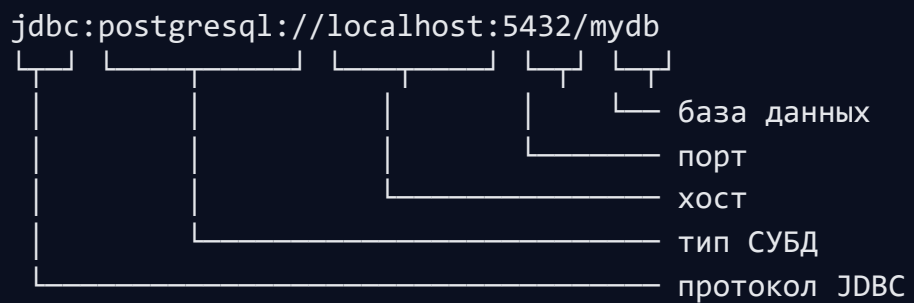


Основные классы JDBC

Класс	Назначение
DriverManager	Управление драйверами, получение Connection
Connection	Соединение с БД
Statement	Выполнение простого SQL
PreparedStatement	SQL с параметрами (используй это!)
ResultSet	Результат SELECT запроса

URL подключения

```
jdbc:postgresql://localhost:5432/mydb
```



база данных
порт
хост
тип СУБД
протокол JDBC

Примеры для разных БД:

```
PostgreSQL: jdbc:postgresql://localhost:5432/mydb  
MySQL:      jdbc:mysql://localhost:3306/mydb  
H2 (memory): jdbc:h2:mem:testdb  
H2 (file):   jdbc:h2:file:./data/mydb
```



Подключение к БД

```
// URL, пользователь, пароль
String url = "jdbc:postgresql://localhost:5432/mydb";
String user = "postgres";
String password = "secret";

// Получение соединения
Connection connection = DriverManager.getConnection(url, user, password);

// Работа с БД...

// ВАЖНО: закрыть соединение!
connection.close();
```



try-with-resources

```
//  Соединение закроется автоматически
try (Connection conn = DriverManager.getConnection(url, user, password)) {

    // Работа с базой данных

} // conn.close() вызовется автоматически, даже при исключении!
```

Connection implements AutoCloseable

```
try (Connection conn = ...;
     Statement stmt = ...;
     ResultSet rs = ...) {
    // Все три ресурса закроются в обратном порядке
}
```

Простой SELECT

```
public List<User> findAllUsers() {
    List<User> users = new ArrayList<>();
    String sql = "SELECT id, name, email FROM users";

    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
            User user = new User();
            user.setId(rs.getLong("id"));
            user.setName(rs.getString("name"));
            user.setEmail(rs.getString("email"));
            users.add(user);
        }

    } catch (SQLException e) {
        throw new RuntimeException("Ошибка при получении", e);
    }

    return users;
}
```

SQL Injection — ОПАСНО!

```
// ❌ НИКОГДА ТАК НЕ ДЕЛАЙ!  
String userInput = request.getParameter("email");  
String sql = "SELECT * FROM users WHERE email = '" + userInput + "'";
```

Что если пользователь введёт:

```
' OR '1'='1
```

Запрос превратится в:

```
SELECT * FROM users WHERE email = ' ' OR '1'='1'
```

Вернёт ВСЕХ пользователей! 



PreparedStatement — защита!

```
public User findByEmail(String email) {  
    // Параметр обозначается ?  
    String sql = "SELECT id, name, email FROM users WHERE email = ?";  
  
    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);  
        PreparedStatement pstmt = conn.prepareStatement(sql)) {  
  
        // Безопасная подстановка параметра (индексация с 1!)  
        pstmt.setString(1, email);  
  
        try (ResultSet rs = pstmt.executeQuery()) {  
            if (rs.next()) {  
                User user = new User();  
                user.setId(rs.getLong("id"));  
                user.setName(rs.getString("name"));  
                return user;  
            }  
        }  
    } catch (SQLException e) {  
        throw new RuntimeException("Ошибка", e);  
    }  
    return null;  
}
```



INSERT с возвратом ID

```
public Long createUser(User user) {
    String sql = "INSERT INTO users (name, email) VALUES (?, ?)";

    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql,
            Statement.RETURN_GENERATED_KEYS)) { // Флаг!

        pstmt.setString(1, user.getName());
        pstmt.setString(2, user.getEmail());
        pstmt.executeUpdate(); // Не executeQuery!

        // Получаем сгенерированный ID
        try (ResultSet keys = pstmt.getGeneratedKeys()) {
            if (keys.next()) {
                return keys.getLong(1);
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Ошибка", e);
    }
    return null;
}
```



UPDATE и DELETE

```
public boolean updateUser(Long id, String newName) {
    String sql = "UPDATE users SET name = ?, updated_at = NOW() WHERE id = ?";

    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, newName);
        pstmt.setLong(2, id);

        int affectedRows = pstmt.executeUpdate();
        return affectedRows > 0; // true если обновили хотя бы одну строку

    } catch (SQLException e) {
        throw new RuntimeException("Ошибка", e);
    }
}
```

DELETE аналогично — просто другой SQL.

Транзакции

Проблема: перевод денег

```
// Шаг 1: списать с одного счёта
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;

// Шаг 2: зачислить на другой
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
```

Что если между шагами произойдёт сбой? 💀

Деньги списаны, но не зачислены!



Транзакции в JDBC

```
Connection conn = null;
try {
    conn = DriverManager.getConnection(URL, USER, PASS);
    conn.setAutoCommit(false); // Начинаем транзакцию

    // Операции внутри транзакции...
    executeWithdraw(conn, fromAccount, amount);
    executeDeposit(conn, toAccount, amount);

    conn.commit(); // Фиксируем — всё успешно
} catch (SQLException e) {
    if (conn != null) {
        conn.rollback(); // Откатываем — что-то пошло не так
    }
    throw new RuntimeException("Ошибка перевода", e);
} finally {
    if (conn != null) {
        conn.setAutoCommit(true);
        conn.close();
    }
}
```



Batch операции

```
public void insertUsers(List<User> users) {
    String sql = "INSERT INTO users (name, email) VALUES (?, ?)";

    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        conn.setAutoCommit(false);

        for (User user : users) {
            pstmt.setString(1, user.getName());
            pstmt.setString(2, user.getEmail());
            pstmt.addBatch(); // Добавляем в очередь
        }

        int[] results = pstmt.executeBatch(); // Выполняем всё разом
        conn.commit();

        System.out.println("Вставлено: " + results.length);
    } catch (SQLException e) {
        throw new RuntimeException("Ошибка batch insert", e);
    }
}
```

⚠ Подводная камень #1: Утечка соединений

```
// ❌ Connection не закроется при исключении!  
Connection conn = DriverManager.getConnection(...);  
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery(sql);  
// Если здесь исключение – ресурсы утекают!  
rs.close();  
stmt.close();  
conn.close();
```

```
// ✅ try-with-resources закроет всё автоматически  
try (Connection conn = DriverManager.getConnection(...);  
    Statement stmt = conn.createStatement();  
    ResultSet rs = stmt.executeQuery(sql)) {  
    // работа с данными  
} // Закроется даже при исключении!
```

⚠ Подводная камень #2: Индексация с 1

```
// ❌ SQLException! В JDBC индексация с 1, не с 0!  
pstmt.setString(0, value);
```

```
// ✅ Первый параметр – индекс 1  
pstmt.setString(1, value);
```

```
// Аналогично для ResultSet  
rs.getString(1); // Первый столбец  
rs.getLong(2);  // Второй столбец
```

Это legacy-решение из 90-х 🦖

⚠ Подводная камень #3: Connection на каждый запрос

```
// ❌ Создание соединения — дорогая операция!  
public User findById(Long id) {  
    try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {  
        // Запрос...  
    }  
}  
// Вызывается 1000 раз = 1000 TCP handshakes!
```

Решение: Connection Pool

```
// ✅ HikariCP — быстрый пул соединений  
HikariDataSource ds = new HikariDataSource();  
ds.setJdbcUrl(URL);  
ds.setMaximumPoolSize(10);  
  
// Получение соединения из пула — микросекунды  
try (Connection conn = ds.getConnection()) {  
    // ...  
}
```



Best Practice: Обработка SQLException

```
catch (SQLException e) {  
    // Логируем детали для отладки  
    log.error("SQL Error - State: {}, Code: {}, Message: {}",  
        e.getSQLState(),    // Стандартный код ошибки  
        e.getErrorCode(),   // Код ошибки БД  
        e.getMessage());  
  
    // Преобразуем в unchecked exception  
    throw new DataAccessException("Ошибка доступа к данным", e);  
}
```

SQLState коды (примеры):

- 23505 — нарушение уникальности
- 23503 — нарушение внешнего ключа
- 42P01 — таблица не существует

Best Practice: Логирование SQL

```
# application.properties (для Spring Boot)
logging.level.org.springframework.jdbc=DEBUG

# Или для чистого JDBC — используй p6spy
```

В development — полезно видеть:

```
DEBUG: Executing SQL query [SELECT * FROM users WHERE id = ?]
DEBUG: Setting parameter 1 to [42]
DEBUG: Query returned 1 row in 3 ms
```



Практическое задание

Создай **UserDao** (Data Access Object):

```
public interface UserDao {  
    User findById(Long id);  
    Optional<User> findByEmail(String email);  
    List<User> findAll();  
    Long create(User user);  
    boolean update(User user);  
    boolean delete(Long id);  
}
```

Реализуй JdbcUserDao:

- PreparedStatement для всех операций
- try-with-resources
- Правильная обработка ошибок



Критерии выполнения

- [] Все CRUD методы работают
- [] Используется **PreparedStatement** (не Statement)
- [] Ресурсы закрываются корректно (try-with-resources)
- [] Нет возможности **SQL Injection**
- [] SQLException правильно обрабатывается
- [] Есть тесты с **H2 in-memory** БД

Бонус ★

- Добавь метод `findByEmailPattern(String pattern)` с LIKE
- Реализуй `batchCreate(List<User> users)`
- Добавь транзакцию для создания User + Profile

Резюме

Тема	Ключевой момент
Connection	Ресурс, требует закрытия
try-with-resources	Автоматическое закрытие
PreparedStatement	Защита от SQL injection
executeQuery()	Для SELECT
executeUpdate()	Для INSERT/UPDATE/DELETE
Транзакции	setAutoCommit(false), commit/rollback
Batch	addBatch() + executeBatch()
Connection Pool	HikariCP для production



Ты освоил JDBC

Следующая тема:

Spring JdbcTemplate

- -
 -
 -
- Меньше бойлерплейта
RowMapper
NamedParameterJdbcTemplate
Интеграция со Spring